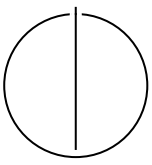
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Computational Science and Engineering

Fast Multipole Boundary Element Method for Finite Periodic Structures

Abhijeet Abhay Sutar



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

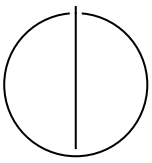
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Computational Science and Engineering

Fast Multipole Boundary Element Method for Finite Periodic Structures

Titel der Abschlussarbeit

Author:	Abhijeet Abhay Sutar
Examiner:	Steffen Marburg, Prof. Dr.
Supervisor:	Arne Hildenbrand, M.Sc.
Submission Date:	Submission date



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Abhijeet Abhay Sutar

Acknowledgments

Abstract

Contents

1 Introduction

1.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

For more details, see the documentation of the `acronym` package[1].

1.1.1 Subsection

See Table 1.1, Figure 1.1, Figure 1.2, Figure 1.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

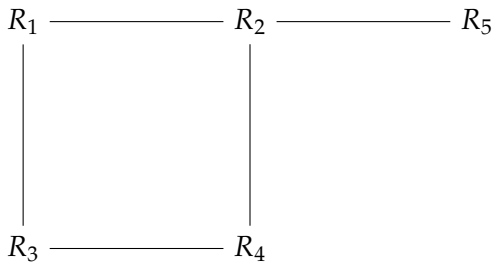


Figure 1.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

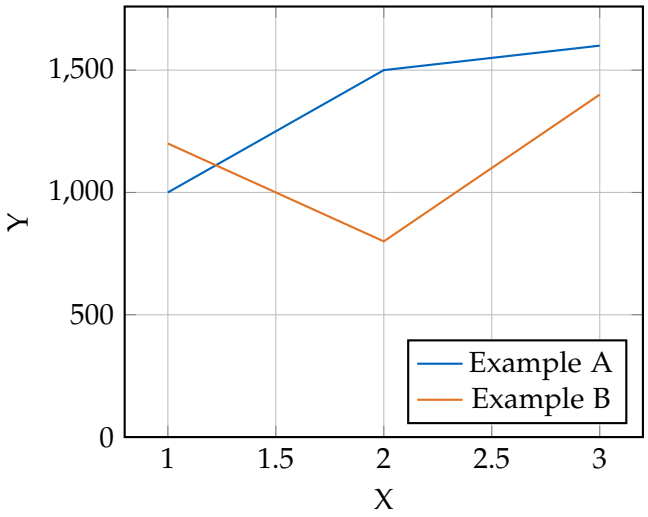


Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Mathematical Formulation of BEM

Before we discus the implementation of the Boundary Element Method (BEM) for the Acoustic Helmholtz equation, we first need

2.1 Acoustic Wave Equation

The propagation of acoustic waves in a homogeneous, inviscid fluid medium is governed by the linear wave equation for the acoustic pressure $p(\mathbf{x}, t)$:

$$\nabla^2 p(\mathbf{x}, t) - \frac{1}{c^2} \frac{\partial^2 p(\mathbf{x}, t)}{\partial t^2} = 0 \tag{2.1}$$

where $\mathbf{x}$ is the position vector, t is time, and c is the speed of sound in the medium.

To derive the Helmholtz equation, we consider a time-harmonic acoustic field. We assume the pressure can be represented as a complex function oscillating with angular frequency ω:

$$p(\mathbf{x}, t) = P(\mathbf{x}) e^{-i\omega t} \tag{2.2}$$

where $P(\mathbf{x})$ is the spatial component of the pressure (the phasor) and $i = \sqrt{-1}$ is the imaginary unit.

Substituting Equation (2.2) into the wave equation (2.1):

$$\nabla^2 \left(P(\mathbf{x}) e^{-i\omega t} \right) - \frac{1}{c^2} \frac{\partial^2}{\partial t^2} \left(P(\mathbf{x}) e^{-i\omega t} \right) = 0 \tag{2.3}$$

Calculating the time derivatives:

$$\frac{\partial}{\partial t} \left(P(\mathbf{x}) e^{-i\omega t} \right) = -i\omega P(\mathbf{x}) e^{-i\omega t} \tag{2.4}$$

$$\frac{\partial^2}{\partial t^2} \left(P(\mathbf{x}) e^{-i\omega t} \right) = (-i\omega)^2 P(\mathbf{x}) e^{-i\omega t} = -\omega^2 P(\mathbf{x}) e^{-i\omega t} \tag{2.5}$$

Substituting this back into the wave equation:

$$e^{-i\omega t} \nabla^2 P(\mathbf{x}) - \frac{1}{c^2} (-\omega^2 P(\mathbf{x}) e^{-i\omega t}) = 0 \tag{2.6}$$

Dividing by the common time factor $e^{-i\omega t}$ (which is non-zero) and rearranging the terms:

$$\nabla^2 P(\mathbf{x}) + \frac{\omega^2}{c^2} P(\mathbf{x}) = 0 \tag{2.7}$$

Introducing the wavenumber $k = \omega/c$, we arrive at the homogeneous Helmholtz equation:

$$\nabla^2 P(\mathbf{x}) + k^2 P(\mathbf{x}) = 0 \tag{2.8}$$

2.2 Boundary Element Method

2.2.1 Boundary Integral Formulation

The Boundary Element Method (BEM) solves the Helmholtz equation by transforming it into an integral equation defined on the boundary of the domain. This transformation is typically achieved using Green's Second Identity.

Let Ω be the domain containing the fluid and Γ be its boundary. For two scalar fields P and G in Ω, Green's Second Identity states [Wu00]:

$$\int_\Omega \left(P\nabla^2 G - G\nabla^2 P\right) d\Omega = \int_\Gamma \left(P\frac{\partial G}{\partial n} - G\frac{\partial P}{\partial n}\right) d\Gamma \tag{2.9}$$

where n is the outward unit normal vector to the boundary.

We choose $P(\mathbf{x})$ to be the acoustic pressure, satisfying the Helmholtz equation derived in Equation (2.8):

$$\nabla^2 P(\mathbf{x}) + k^2 P(\mathbf{x}) = 0 \tag{2.10}$$

We choose $G(\mathbf{x}, \mathbf{y})$ to be the fundamental solution (Green's function) of the Helmholtz equation, which satisfies the inhomogeneous equation with a point source at $\mathbf{y}$:

$$\nabla^2 G(\mathbf{x}, \mathbf{y}) + k^2 G(\mathbf{x}, \mathbf{y}) = -\delta(\mathbf{x} - \mathbf{y}) \tag{2.11}$$

where δ is the Dirac delta function. For 3D acoustics, the free-space Green's function satisfying the Sommerfeld radiation condition is given by [Kir98]:

$$G(\mathbf{x}, \mathbf{y}) = \frac{e^{ikr}}{4\pi r} \quad \text{with} \quad r = |\mathbf{x} - \mathbf{y}| \tag{2.12}$$

Substituting these into Green's Second Identity for a source point $\mathbf{y}$ inside the domain Ω:

$$P(\mathbf{y}) = \int_\Gamma \left(G(\mathbf{x},\mathbf{y}) \frac{\partial P}{\partial n}(\mathbf{x}) - P(\mathbf{x}) \frac{\partial G}{\partial n}(\mathbf{x},\mathbf{y}) \right) d\Gamma(\mathbf{x}) \tag{2.13}$$

As the source point $\mathbf{y}$ is moved to the boundary Γ, the integrals become singular. Evaluating the singularity using a limiting process yields the Boundary Integral Equation (BIE) [Mar08]:

$$C(\mathbf{y})P(\mathbf{y}) = \int_\Gamma \left(G(\mathbf{x},\mathbf{y}) \frac{\partial P}{\partial n}(\mathbf{x}) - P(\mathbf{x}) \frac{\partial G}{\partial n}(\mathbf{x},\mathbf{y}) \right) d\Gamma(\mathbf{x}) \tag{2.14}$$

where $C(\mathbf{y})$ is the geometric coefficient (solid angle). For a smooth boundary, $C(\mathbf{y}) = 0.5$.

2.3 Discretization

To solve the BIE numerically, the boundary Γ is discretized into N boundary elements. The pressure P and its normal derivative $q = \partial P/\partial n$ are approximated using shape functions.

Applying the collocation method at the nodes leads to a system of linear algebraic equations:

$$\mathbf{HP} = \mathbf{GQ} \tag{2.15}$$

where $\mathbf{H}$ and $\mathbf{G}$ are the dense, nonsymmetric influence matrices resulting from the integration of the fundamental solution and its derivative, and $\mathbf{P}$ and $\mathbf{Q}$ are vectors containing the nodal values of pressure and normal velocity, respectively.

2.4 Burton-Miller Formulation

The Boundary Element Method (BEM) offers a highly elegant, mathematically rigorous, and computationally efficient alternative by reformulating the volumetric partial differential equation into a Boundary Integral Equation (BIE) mapped strictly over the surface of the scattering or radiating body. This approach intrinsically satisfies the Sommerfeld radiation condition at infinity, thereby reducing the dimensionality of the problem by one—transforming a three-dimensional volumetric domain into a two-dimensional surface mesh, or a two-dimensional problem into a one-dimensional contour [Lam94; Wu00; Mar08; Kir98].

However, as [BM71] showed, the standard BEM approach for exterior problems, gives rise to non-existence of unique solutions at wavenumbers coinciding with the resonances of the corresponding interior problem.

At these critical frequencies, also sometime referred to as *fictitious or irregular frequencies*, the matrix system resulting from the discretization of the BIE becomes ill-conditioned, leading to numerical instability and a spike in the error of the computed solution.

It must be noted that the original exterior problem does have a unique solution at these frequencies and that the connection to the interior problem has no physical significance. The non-uniqueness arises solely from the formulation of the problem in terms of a Boundary Integral Equation (BIE).[BM71]

3 The Fast Multipole Method Framework

The BEM lends itself naturally to modelling acoustic wave propagation. Transforming the volumetric governing differential equations into boundary integral equations allows finite structures to be treated in an infinite exterior domain without requiring special boundary conditions. However, the required mathematical transformation of the problem yields a full, complex, non-Hermitian matrix [CL07]. As the degrees of freedom increase to model larger problem, the memory requirements to hold the large full matrix and the cost to compute the matrix coefficients themselves, scale as $\mathcal{O}(N^2)$, and the computational cost of solving the resulting linear system using direct solvers scales as $\mathcal{O}(N^3)$, where N is the number of degrees of freedom [Liu09]. This makes the BEM computationally prohibitive for large-scale problems.

To overcome this computational bottleneck, we must employ a radically different numerical strategy. This chapter details the algorithmic design and the mathematical formulation of the FMM framework. The FMM decouples far-field interactions through hierarchical spatial decomposition, truncated analytical basis expansions, and the application of complex mathematical translation operators [CRW93].

This chapter focuses on the mathematical and algorithmic design of the FMM implemented as a part of this thesis. It details the division of the mesh space into a topological tree data structure, the mathematics of the multi-stage multipole translations, and the implementation of near-field interactions.

3.1 Fast Multipole Method

The FMM was introduced in 1987 by Greengard and Rokhlin [GR87] to accelerate large-scale particle simulations involving long-range pairwise potential interactions. Later, it was extended to other problems, including the BEM for Acoustic Scattering [Rok90]. The FMM handles short-range interactions with direct computation, and uses analytical multipole expansions to compute potentials for long-range interactions. By providing a method for the rapid computation of the matrix-vector product between the system matrix any arbitrary vector, the FMM enables the use of iterative solvers for solving the linear systems arising from the BEM discretization, reducing the computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N^{\frac{3}{2}})$ or even $\mathcal{O}(N \log N)$ for a multilevel implementation [CRW93].

3.1.1 The FMM algorithm

The FMM follows these basic steps:

Step 1. Discretization Any problem begins with discretizing the boundary, as is done for conventional BEM. We use constant elements.

Step 2. Initializing the Octree For a 3D problem, we consider a cube that contains the entire geometry. This cube is repeatedly subdivided into 8 smaller cubes, called its child nodes, creating an octree structure. The subdivision continues until the number of elements in a cube falls below a defined threshold.

Step 3. Upward Pass Beginning from the bottom of the tree, at the deepest level called the leaf nodes, we compute the moments of each node, tracing the tree structure upwards till we are two levels below the root node. Using the Multipole-to-Multipole (M2M) translation operator, the moments of the child nodes are translated and aggregated to the parent node.

Step 4. Downward Pass Starting from the second level of the tree, we compute the local expansions for each node. This consists of contributions from the current node's interaction list and from far-off nodes. The interaction list nodes are those not directly adjacent to the current node, but whose parent node is adjacent to the current node's parent. The contribution from the interaction list nodes is computed using the M2L translation operator, while the contribution from the far-off nodes is computed using the L2L translation operator for the parent node, with the expansion point being shifted from the centroid of the parent node to the centroid of the current node. At level 2, the nodes only have contributions from the interaction list nodes. Since there are no far-off nodes at this level, we use the M2L operator to get the local expansion's coefficients.

Step 5. Evaluating of the Integrals The contributions of all the collocation points in the current node and thed nodes directly adjacent to it are computed using direct BEM. The contributions of all nodes in the interaction list and the far-off are computed using the local expansion coefficients of the current node and shifting the expansion point to the collocation point using the Local-to-Particle (L2P) translation operator.

Step 6. Iterative Solver We update the unknown vector, in this case the `phi` vector in the system of equations $\mathbf{A}\boldsymbol{\phi} = \mathbf{b}$, using an iterative solver, such as Generalized Minimal Residual Method (GMRES), and repeat from steps 3 until convergence is achieved.

3.2 Octree

The spatial domain can be subdivided into a hierarchical tree structure in many ways. For 3D problems, the most common approach is to use an octree, which offers many opportunities to optimize algorithmic efficiency. An octree is a tree data structure where each node is bisected along the Cartesian x, y, and z axes, producing 8 equally sized child nodes, and these child nodes are then further subdivided into 8 nodes until a certain threshold is reached.

3.2.1 Nodes of the Octree

We define an `FMMNode` data structure to represent each node in the octree, which contains the following information:

Node geometry: The struct keeps track of the node's bounding box, its center, and its radius. It also stores the integer grid coordinates that represent the node's position in the octree.

Tree Hierarchy: Each node is assigned a node index, which serves as the node's unique identifier within a global list of all the nodes. The node also stores its tree level and the index of its parent node. Most importantly, it stores the indices of its 8 child nodes, which are used to navigate the tree.

Mesh and Interaction data: The node stores an iterator for the particles contained within it. It maintains a neighbor list that contains the indices of neighboring nodes. These interactions must be computed directly. It also stores a list of far-field nodes, which are sufficiently far away to be approximated by multipole expansions.

Mathematical expansion data: The node stores `Pm` which is the order used for the mutlitpole expansion, and the number of coefficients required based on the order. It also stores the multipole and local expansion coefficients, which are used to approximate the potential and forces from distant nodes.

FMM Translation Operators: The node stores the necessary translation operators for the FMM algorithm.

1. Particle-to-Multipole (P2M) Matrix: Translates raw particles (mesh sources) to a node's Multipole expansion.
2. M2M Matrix: Translates a child's Multipole expansion up to its parent's Multipole expansion.

3. Multipole-to-Local (M2L) Matrices: Converts a distant node's Multipole expansion into this node's Local expansion.
4. Local-to-Local (L2L) Matrix: Translates a parent's Local expansion down to its child's Local expansion.
5. L2P Matrix: Evaluates the Local expansion directly onto the target particles (mesh cells).

3.2.2 Splitting the domain to make an Octree

The underlying spatial data-structue sets the stage for the computational efficiency of the FMM algorithm. It dictates how the source-target interactions are categorized as either near-field, needing direct computation, or far-field, and thus suitable for approximation via multipole expansions. An adaptive, heirarchical octree allows for a systematic way to manage these interactions.

Space-Filling Curves and Morton Encoding

To facilitate efficient hierarchical clustering and rapid neighborhood queries within the computational grid, the geometric centers of the boundary elements are mapped from three-dimensional space into a one-dimensional array using a space-filling curve, specifically, the Morton Z-curve. This encoding sorts mesh elements by their spatial coordinates so that elements that are close in space remain close in memory, improving cache locality. While the Morton Z-order curve is not the optimum space-filling curve for 3D problems, its simplicity of implementation via bitwise operations lends itself to efficient execution [MB15].

Morton encoding converts the 3D coordinated of each mesh element into single integer by interleaving the bits of the binary representations of the x, y, and z coordinates, generating that element's Morton key. This ordering assigns numerically adjacent Morton keys to spatially adjacent elements, inherently preserving spatial locality. This allows us to then sort the elements into a linearly ordered array, where adjacent elements are already clustered together, which helps us efficiently assign elements to the octree's nodes.

4 Software Architecture and HPC Optimizations

4.1 Bitwise operations for Morton Encoding

As detailed in Section **??**, the Morton Z-order curve is used to map the 3D coordinates of the mesh elements for a more effient octree construction. To optimize this process without relying on expensive looping, we use a series of bitwise operations that interleave the bits of the binary representations of the x, y, and z coordinates.

The basic algorithm that we use to compute the Morton key for a given 3D coordinate is as follows:

Step 1. **Grid Quantization:** We define a discrete grid across the global bounding box. With a 64 bit integer, we can divide each spatial dimension into 2^{20} $(1,048,576)$ bins, which allows us to represent up to 2^{60} unique spatial locations, more than enough for our problem size.

Step 2. **Coordinate Normalization:** We normalize the x, y, and z coordinates of each mesh element to fit within $[0,1]$ relative to the global bounding box, and then multiply by the number of bins to get the quantized integer coordinates.

Step 3. **Bit Truncation:** We truncate the quantized coordinates to fit within 21 bits, ensuring that they can be interleaved into a single 64-bit integer without overflow. This is achieved by applying a bitwise AND operation with a mask that retains only the lower 21 bits of each coordinate.

Step 4. **Bit Expansion:** We use a sequence of bitwise shifts and bitwise operations with bit masks to add two zero bits between each bit of the quantized coordinates. This process is repeated for each coordinate.

Step 5. **Bit Interleaving:** Finally, we combine the expanded bits of each coordinate by shifting the expanded y bits by one, and the expanded z bits by two, and then performing a bitwise OR operation to interleave the bits together, resulting in the final Morton key for that mesh element.

The basics algorithm of this process is shown for the 2D case in [And05]

4.2 Caching the Matrices

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.3 OpenMP speed up

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.4 Solving the system - vmult + GMRES + The Real-Equivalent system

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo

ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

5 Performance Analysis and Validation

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

6 Conclusion and Future Directions

- Currently Burton - Miller is highly optimized for single dof per cell and for perfectly flat panels. That is, FE_Degree = 0 and Mapping_Degree = 1. To support higher degrees, we need to update the hypersingular calculation in the near field operator and the main assemble system method.
- There is some basic parallelization in the code for computing the translation matrices and for the matrix-vector multiplication done with OpenMP pragmas. But, Deal.ii has its native parallelization support using its native datatypes. The FMM also lends itself to parallelization where each node of the FMM tree may reside in an individual process and maintain its own near field matrix, with only the far-off node dipoles having to be passed around.
- The current implementation of the computation of the external field uses drect computations of the green's function. This can be upgraded to use the FMM approximation in the future, which will be faster for large problems.
- The current implementation is strictly for scattering off of sound hard objects. The next step would be to implment the option to set an arbitrary impedance boundary condition.
- The current implementation is strictly for a stationary mesh.

Abbreviations

BEM Boundary Element Method

FMM Fast Multipole Method

BIE Boundary Integral Equation

M2M Multipole-to-Multipole

M2L Multipole-to-Local

L2L Local-to-Local

L2P Local-to-Particle

P2M Particle-to-Multipole

GMRES Generalized Minimal Residual Method

List of Figures

List of Tables

Bibliography

[And05] S. E. Anderson. *Bit Twiddling Hacks*. Bit Twiddling Hacks. May 5, 2005. URL: https://graphics.stanford.edu/~seander/bithacks.html#InterleaveBMN (visited on 03/21/2026).

[BM71] A. J. Burton and G. F. Miller. "The Application of Integral Equation Methods to the Numerical Solution of Some Exterior Boundary-Value Problems." In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 323.1553 (June 8, 1971), pp. 201–210. ISSN: 0080-4630, 2053-9169. DOI: 10.1098/rspa.1971.0097.

[CL07] S. Chandler-Wilde and S. Langdon. "Boundary Element Methods for Acoustics." In: (July 18, 2007).

[CRW93] R. Coifman, V. Rokhlin, and S. Wandzura. "The Fast Multipole Method for the Wave Equation: A Pedestrian Prescription." In: *IEEE Antennas and Propagation Magazine* 35.3 (June 1993), pp. 7–12. ISSN: 1045-9243. DOI: 10.1109/74.250128.

[GR87] L. Greengard and V. Rokhlin. "A Fast Algorithm for Particle Simulations." In: *Journal of Computational Physics* 73.2 (Dec. 1987), pp. 325–348. ISSN: 00219991. DOI: 10.1016/0021-9991(87)90140-9.

[Kir98] S. Kirkup. *The Boundary Element Method in Acoustics: A Development in Fortran*. Integral Equation Methods in Engineering 1. Hebden Bridge: Integrated Sound Software, 1998. 136 pp. ISBN: 978-0-9534031-0-3.

[Lam94] L. Lamport. *LaTeX : A Documentation Preparation System User's Guide and Reference Manual*. Addison-Wesley Professional, 1994.

[Liu09] Y. Liu. *Fast Multipole Boundary Element Method: THEORY AND APPLICATIONS IN ENGINEERING*. CAMBRIDGE UNIVERSITY PRESS, 2009.

[Mar08] S. Marburg. "Boundary Element Method for Time-Harmonic Acoustic Problems." In: *Computational Acoustics of Noise Propagation in Fluids*. Ed. by S. Marburg and B. Nolte. Berlin, Heidelberg: Springer, 2008.

[MB15] D. Malhotra and G. Biros. "PVFMM: A Parallel Kernel Independent FMM for Particle and Volume Potentials." In: *Communications in Computational Physics* 18.3 (Sept. 2015), pp. 808–830. ISSN: 1815-2406, 1991-7120. DOI: 10.4208/cicp.020215.150515sw.

[Rok90] V. Rokhlin. "Rapid Solution of Integral Equations of Scattering Theory in Two Dimensions." In: *Journal of Computational Physics* 86.2 (Feb. 1990), pp. 414–439. ISSN: 00219991. DOI: 10.1016/0021-9991(90)90107-C.

[Wu00] T. Wu. *Boundary Element Acoustics: Fundamentals and Computer Codes*. Southampton, UK: WIT Press, 2000.